

```
{  
  "status": "active",  
  "data": [ ... ],  
}
```

Praktikum 10: Membangun REST API

Mengkoneksikan Sistem dengan
CodeIgniter 4 & Postman



CodeIgniter 4 Backend

```
{  
  "config": {  
    "author": "active",  
    "variety_aaatd": "..."  
  },  
  "api_version": "v1"  
}
```



Postman API Client

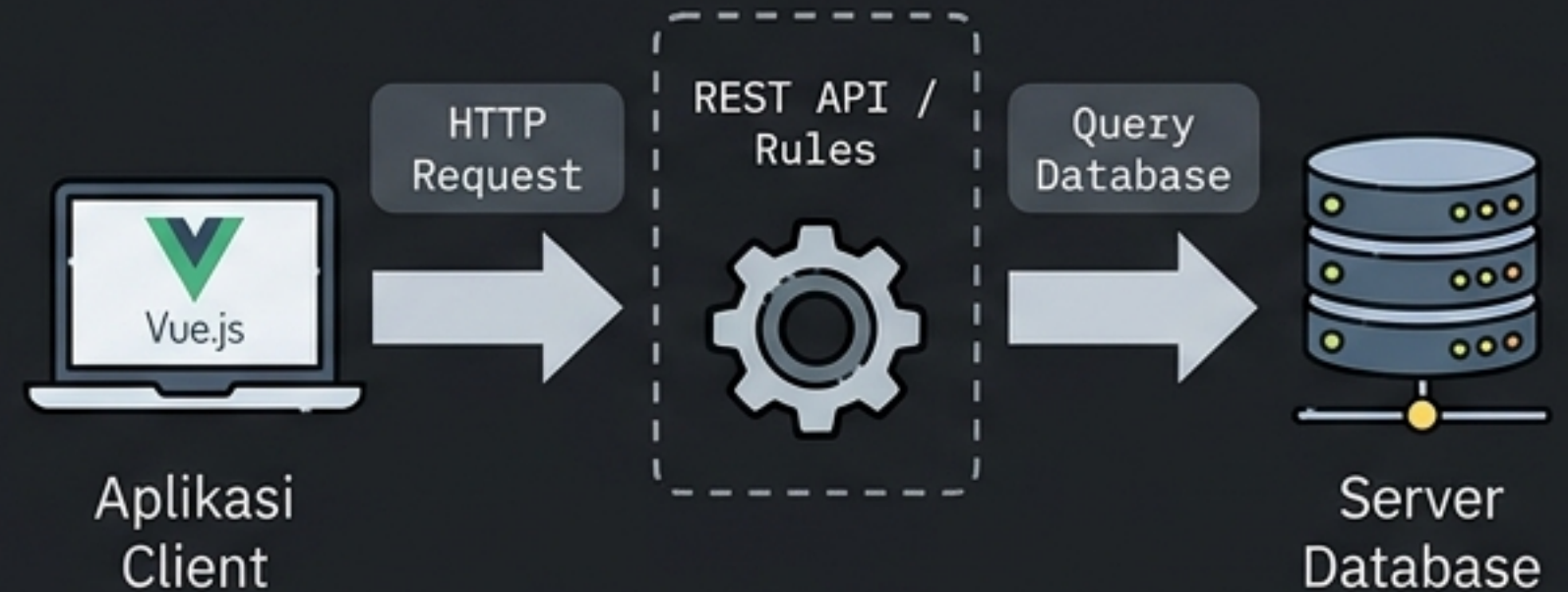
Apa itu REST API?

Dunia Nyata: Restoran



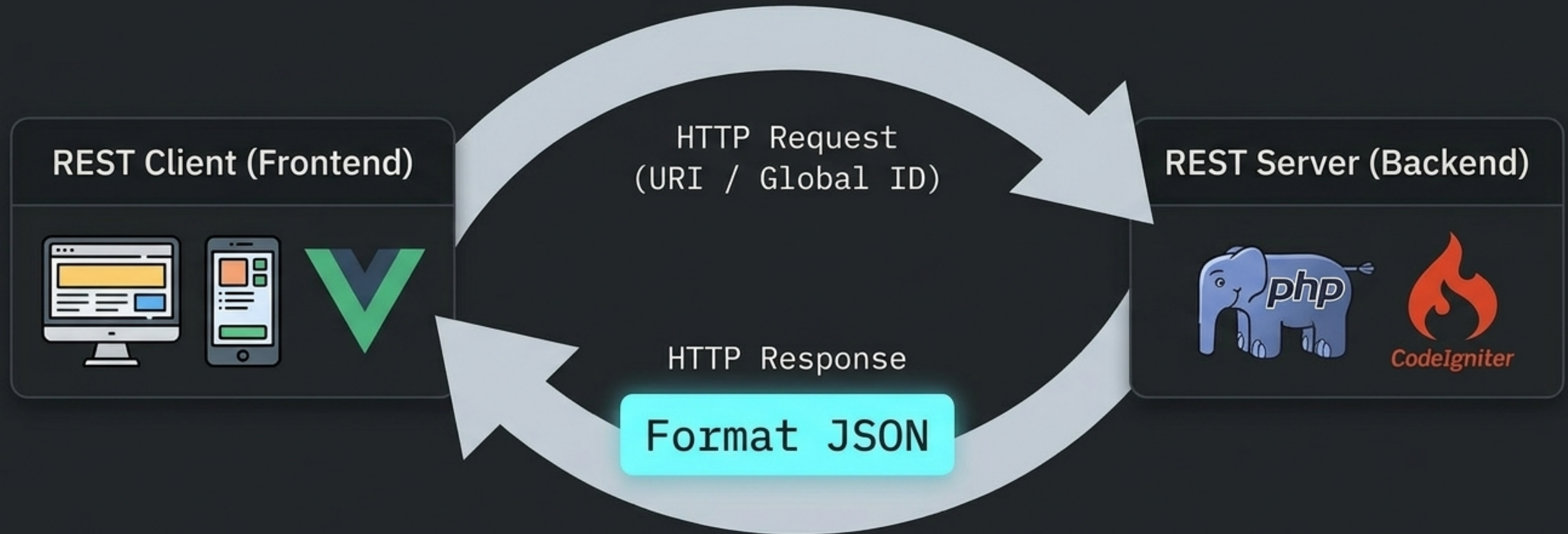
Pelanggan tidak boleh masuk ke dapur. Mereka memesan melalui aturan yang ada di Daftar Menu.

Dunia Digital: Integrasi Sistem



API membatasi hak akses. Aplikasi luar berkomunikasi dengan database Anda hanya melalui titik masuk (endpoint) yang diizinkan.

Arsitektur REST Client-Server



JSON bertindak sebagai bahasa universal. Backend (PHP) dan Frontend (Vue.js) menggunakan bahasa pemrograman berbeda, namun keduanya dapat membaca dan mengirim JSON secara efisien.

Persiapan Lingkungan Kerja



Text Editor

IDE utama untuk menulis dan memodifikasi kode REST Controller.



Webserver & Framework

Direktori 'lab7_php_ci' berjalan pada 'htdocs'. Pastikan local server Anda sudah aktif.



Testing Tool (REST Client)

Aplikasi klien untuk melakukan simulasi dan pengujian endpoint API secara instan tanpa perlu membangun antarmuka frontend.

postman.com/downloads

Matriks Sintesis CRUD & REST

Operasi Database (CRUD)	Metode HTTP	Pola URL (Endpoint)	Fungsi Controller CI4
Read	GET	/post (Semua) atau /post/{id} (Spesifik)	index() & show(\$id)
Create	POST	/post	create()
Update	PUT	/post/{id}	update(\$id)
Delete	DELETE	/post/{id}	delete(\$id)

Hanya dengan 4 metode HTTP standar, kita dapat melakukan seluruh manuver manipulasi data dasar (CRUD).

Membangun Routing Otomatis

app/Config/Routes.php

```
$routes->resource( 'post' );
```

Verifikasi route dengan command: php spark routes

CodeIgniter v4.3.4 Command Line Tool - Server Time: 2023-06-18 06:52:25 UTC+00:00

Method	Route	Name	Handler	Before Filters	After Filters
GET	/	»	\App\Controllers\Home::index		toolbar
GET	post	»	\App\Controllers\Post::index		toolbar
GET	post/new	»	\App\Controllers\Post::new		toolbar
GET	post/(.*)/edit	»	\App\Controllers\Post::edit/\$1		toolbar
GET	post/(.*)	»	\App\Controllers\Post::show/\$1		toolbar
POST	post	»	\App\Controllers\Post::create		toolbar
PATCH	post/(.*)	»	\App\Controllers\Pest::update/\$1		toolbar
PUT	post/(.*)	»	\App\Controllers\Pest::update/\$1		toolbar
DELETE	post/(.*)	»	\App\Controllers\Pest::delete/\$1		toolbar

Sihir CI4: Satu baris kode di atas menghasilkan seluruh variasi endpoint GET, POST, PUT, PATCH, dan DELETE secara otomatis.

Fondasi REST Controller

Controller khusus CI4 untuk menangani struktur RESTful secara native tanpa konfigurasi manual.

```
app/Controllers/Post.php

namespace App\Controllers;

use CodeIgniter\RESTful\ResourceController;
use CodeIgniter\API\ResponseTrait;
use App\Models\ArtikelModel;

class Post extends ResourceController {
    use ResponseTrait;

    // ... metode CRUD diletakkan di sini ...
}
```

Library bantuan esensial untuk merangkai response JSON dengan mudah (misalnya menggunakan `respondCreated` atau `failNotFound`).

Mengimpor model database agar API memiliki akses untuk membaca dan memanipulasi tabel artikel.

Bedah Kodah Kode: Metode Read (GET)

GET All Menampilkan Semua Data

```
public function index() {  
    $model = new ArtikelModel();  
    $data['artikel'] = $model->orderBy('id', 'DESC')->findAll();  
    return $this->respond($data);  
}
```

Mengambil semua baris data artikel, mengurutkan dari yang terbaru, lalu mengubahnya menjadi JSON otomatis via respond().

GET Single Menampilkan Data Spesifik

```
public function show($id = null) {  
    $model = new ArtikelModel();  
    $data = $model->where('id', $id)->first();  
    if ($data) {  
        return $this->respond($data);  
    } else {  
        return $this->failNotFound('Data tidak ditemukan.');    }  
}
```

Cek Data ID

True

`$this->respond($data)`

False

`$this->failNotFound()`

Bedah Kode: Metode Create (POST)

Menangkap Input

```
$data = [  
    'judul' => $this->request->getVar('judul'),  
    'isi'    => $this->request->getVar('isi')  
];
```

Menangkap muatan (payload) yang dikirim oleh REST Client.



Eksekusi Database

```
$model->insert($data);
```

Menyimpan array data ke dalam database MySQL.



Menyusun Response

```
$response = [  
    'status' => 201,  
    'messages' => ['success'  
=> 'Data ditambahkan.']  
];  
return $this->respondCreated($response);
```

Menyiapkan dan mengembalikan respon HTTP 201 Created dengan pesan sukses.

Insight Status Code: Mengapa 201? '201 Created' adalah standar baku HTTP yang secara eksplisit menyatakan bahwa sebuah resource baru telah sukses diciptakan di server.

Bedah Kode: Metode Update (PUT)

Menangkap ID unik sebagai target modifikasi data.

```
public function update($id = null) {  
    $model = new ArtikelModel();  
    $id = $this->request->getVar('id');  
    $data = [  
        'judul' => $this->request->getVar('judul'),  
        'isi'    => $this->request->getVar('isi'),  
    ];  
    $model->update($id, $data);  
    $response = [  
        'status' => 200,  
        'messages' => ['success' => 'Data artikel diubah.']  
    ];  
    return $this->respond($response);  
}
```

Mengeksekusi pembaruan data spesifik di dalam model.

Mengembalikan status '200 OK' sebagai tanda operasi modifikasi telah berhasil dijalankan tanpa hambatan.

Bedah Kode: Metode Delete (DELETE)

```
$data = $model->where('id', $id)->delete($id);
```

Titik Kritis: Apakah data dengan ID tersebut eksis di database?

if (\$data) → Data Ditemukan

else → Data Kosong

```
$model->delete($id);
```

```
return $this->respondDeleted($response);
```

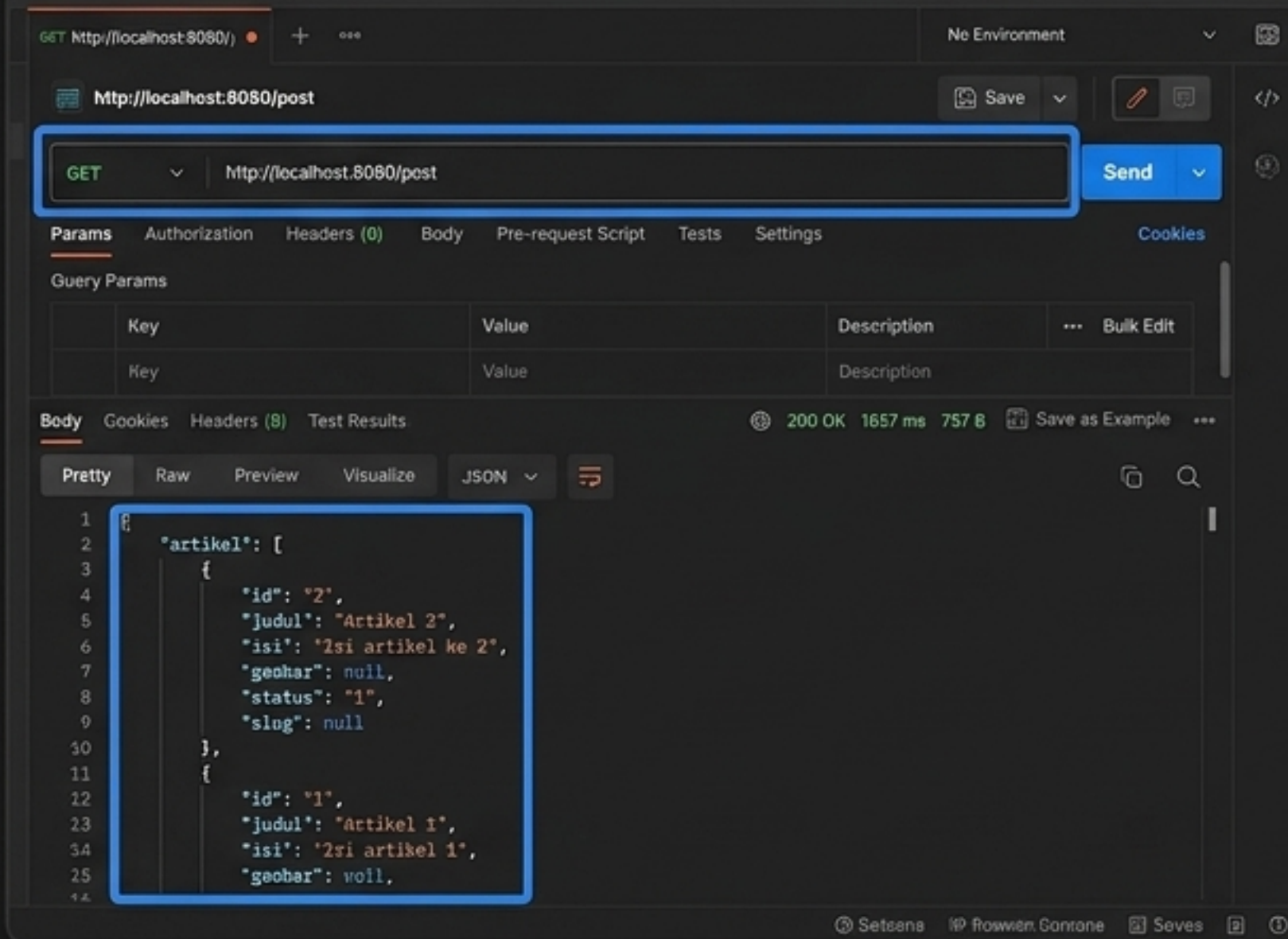
Status Code: 200 OK

```
return $this->failNotFound('Data tidak ditemukan.');
```

Menghentikan proses hapus untuk mencegah error database.

Uji Jalan: Pengujian GET

GET: Semua Data



GET `http://localhost:8080/post`

Send

Params Authorization Headers (0) Body Pre-request Script Tests Settings Cookies

Query Params

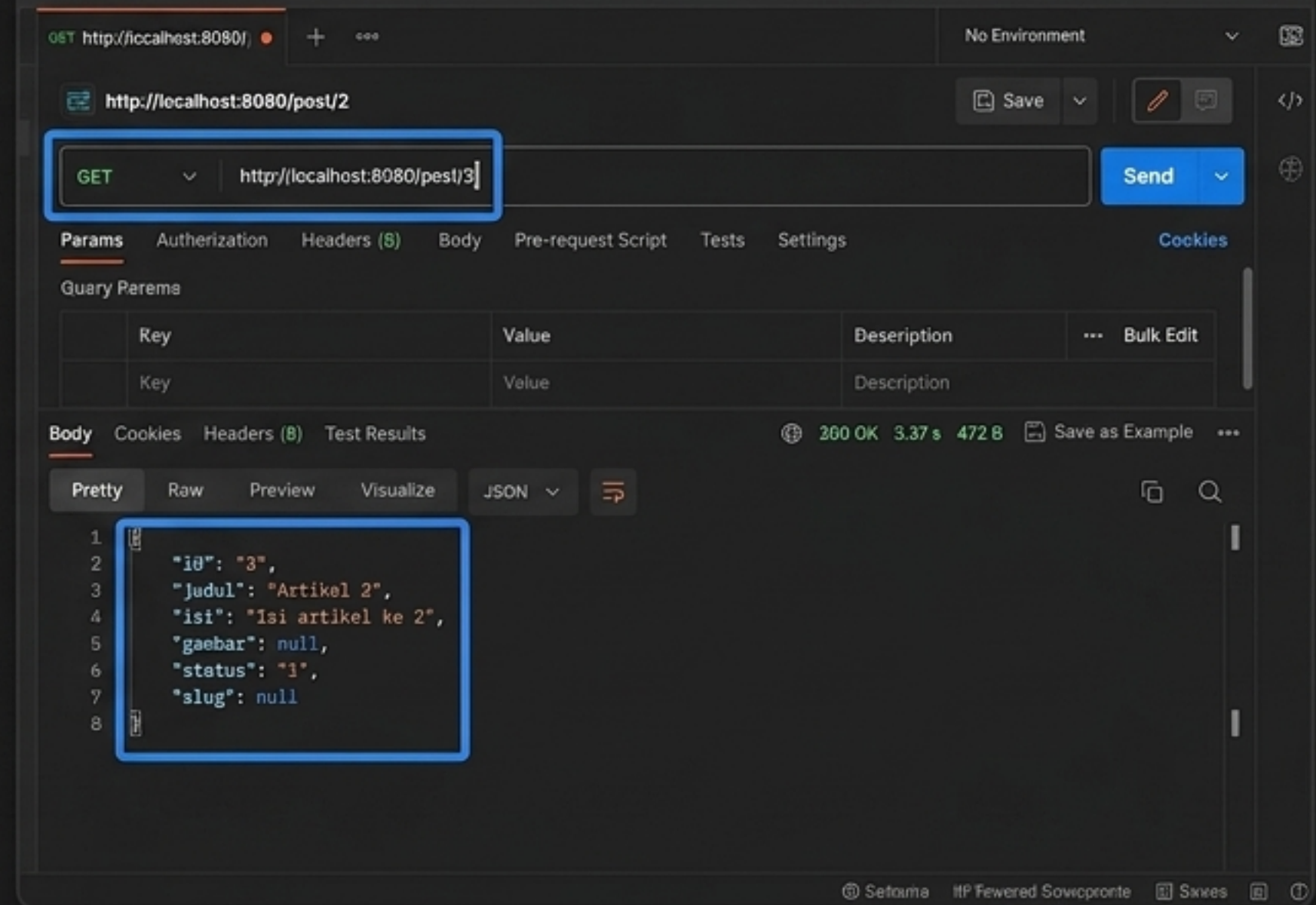
Key	Value	Description	...	Bulk Edit
Key	Value	Description		

Body Cookies Headers (8) Test Results 200 OK 1657 ms 757 B Save as Example

Pretty Raw Preview Visualize JSON

```
1 {
2   "artikel": [
3     {
4       "id": "2",
5       "judul": "Artikel 2",
6       "isi": "Isi artikel ke 2",
7       "gambar": null,
8       "status": "1",
9       "slug": null
10    },
11    {
12      "id": "1",
13      "judul": "Artikel 1",
14      "isi": "Isi artikel ke 1",
15      "gambar": null,
```

GET: Data Spesifik



GET `http://localhost:8080/post/3`

Send

Params Authorization Headers (8) Body Pre-request Script Tests Settings Cookies

Query Params

Key	Value	Description	...	Bulk Edit
Key	Value	Description		

Body Cookies Headers (8) Test Results 200 OK 3.37 s 472 B Save as Example

Pretty Raw Preview Visualize JSON

```
1 {
2   "id": "3",
3   "judul": "Artikel 2",
4   "isi": "Isi artikel ke 2",
5   "gambar": null,
6   "status": "1",
7   "slug": null
8 }
```

Validasi: Jika format JSON muncul di panel Body bagian bawah dengan kode status 200 OK, maka endpoint `index()` dan `show()` telah berfungsi sempurna.

Uji Jalan: Mengirim Data (POST)

Set Method: Ubah dropdown menjadi **POST**, arahkan ke endpoint **/post**.

The screenshot shows the Postman interface for a POST request to `http://localhost:8080/post`. The request is configured with the `x-www-form-urlencoded` body type. The body contains two key-value pairs: `judul` with value `Judul Artikel 4` and `isi` with value `Isi artikel ke 4 ditambah melalui API`. The response is a `201 Created` status with a JSON body indicating success.

Key	Value	Description
judul	Judul Artikel 4	
isi	Isi artikel ke 4 ditambah melalui API	

```
{
  "status": 201,
  "error": null,
  "messages": {
    "success": "Data artikel berhasil ditambahkan."
  }
}
```

Isi Data: Masukkan parameter **KEY** ('judul', 'isi') beserta **VALUE** teks barunya.

Siapkan Payload: Pilih tab Body, lalu aktifkan radio button **x-www-form-urlencoded** untuk mensimulasikan pengiriman form HTML.

Kirim & Analisis: Klik **Send**. Pastikan Anda menerima respon JSON keberhasilan dan status indikator hijau **201 Created**.

Uji Jalan: Modifikasi & Penghapusan

Uji PUT:

Ubah Method menjadi PUT.
Tambahkan ID target pada URL.
Tab Body tetap menggunakan x-www-form-urlencoded untuk mengirim data ubahan.
Cek kembalian status 200 OK.

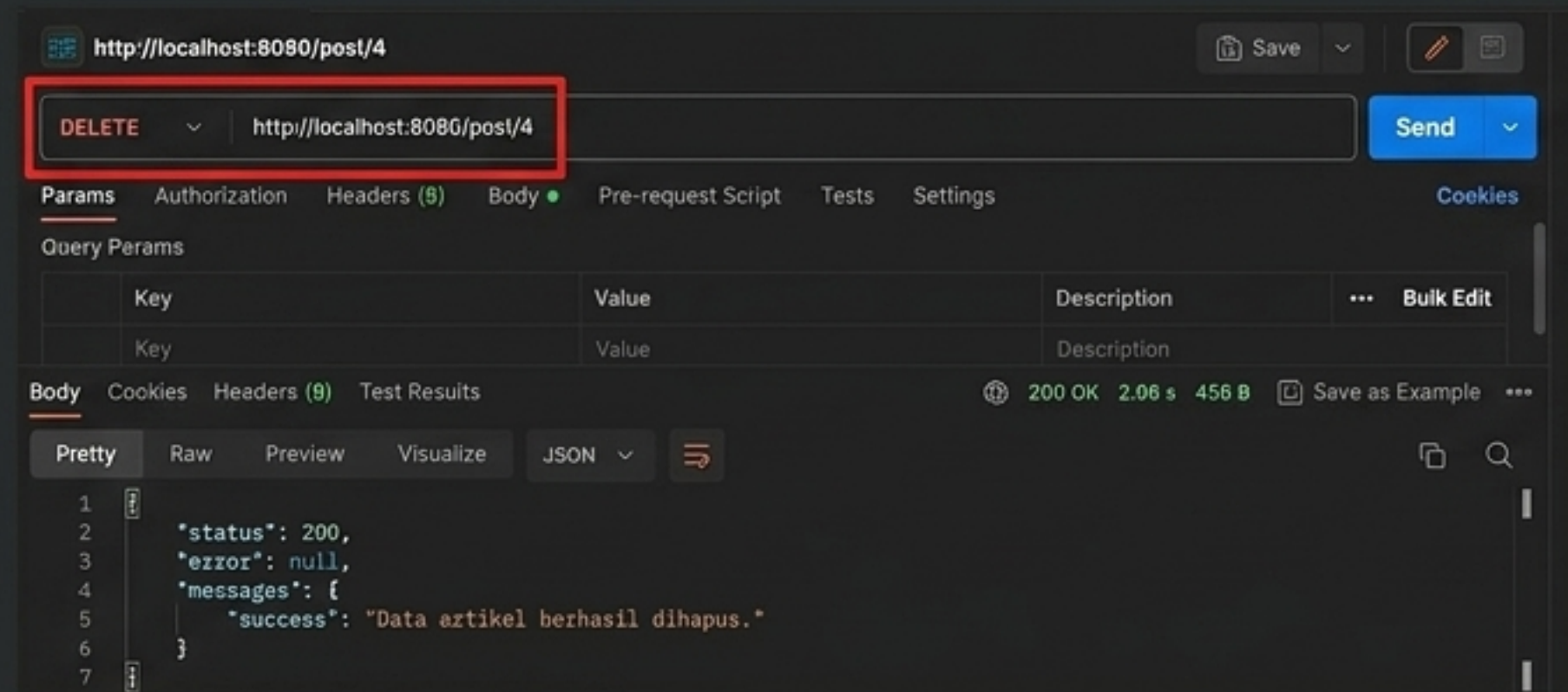


The screenshot shows a Postman PUT request to `http://localhost:8080/post/2`. The request is configured with the method **PUT** and the URL `http://localhost:8080/post/2`. The **Body** tab is selected, showing a table with two rows: **judul** with value `Judul Artikel 2 diubah` and **isi** with value `Isi artikel ke 2 diubah melalui API`. The **Headers** tab shows `x-www-form-urlencoded` selected. The **Test Results** tab shows a status of **200 OK** with a response time of **2.07 s** and a size of **457 B**. The response body is displayed in the **Body** tab, showing a JSON object:

```
{  "status": 200,  "error": null,  "messages": {    "success": "Data artikel berhasil diubah."  }}
```

Uji DELETE:

Ubah Method menjadi DELETE.
Pastikan URL langsung menuju ID target (contoh: `/post/4`). Tidak perlu mengisi payload tab Body.
Cek respons JSON konfirmasi penghapusan.

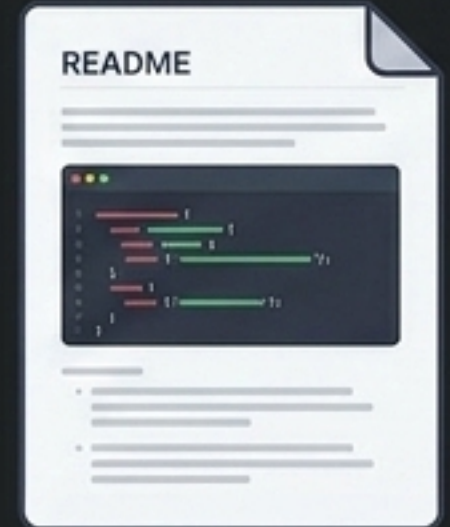


The screenshot shows a Postman DELETE request to `http://localhost:8080/post/4`. The request is configured with the method **DELETE** and the URL `http://localhost:8080/post/4`. The **Body** tab is selected, showing an empty table. The **Headers** tab shows `x-www-form-urlencoded` selected. The **Test Results** tab shows a status of **200 OK** with a response time of **2.06 s** and a size of **456 B**. The response body is displayed in the **Body** tab, showing a JSON object:

```
{  "status": 200,  "error": null,  "messages": {    "success": "Data artikel berhasil dihapus."  }}
```


Deployment & Laporan Akhir

- [] Buka repository lokal dengan nama Lab11Web.
- [] Selesaikan seluruh baris kode dan lakukan pengujian endpoint di Postman sesuai urutan metode.
- [] Ambil screenshot sebagai bukti fungsionalitas dari setiap response Postman.
- [] Perbarui file README.md. Tuliskan dokumentasi teknis dari REST API yang dibangun beserta sisipan screenshot.
- [] Eksekusi Git: Lakukan commit lalu push progres ke repository jarak jauh (GitHub/GitLab).
- [] Salin URL repository dan unggah ke e-learning ecampus sebagai laporan praktikum.



// Sistem terintegrasi. Praktikum selesai. Selamat mengode!